

■■■ SOC Automation Lab

Complete Setup Guide / Guía Completa de Instalación

Suricata IDS · Elasticsearch · Kibana · Filebeat · n8n · Telegram

Stack: Suricata 8.x + ELK Stack 8.x + n8n + Python + Flask + ngrok

Platform: Kali Linux VM (VirtualBox)

Autor / Author: Marcos — @Markussella

GitHub: github.com/Markussella/blue-team-lab

Version: 1.0 — 2026

Built as a hands-on cybersecurity portfolio project — Blue Team / SOC Analyst

■ Table of Contents / Índice

1. Overview / Descripción del Proyecto
2. Architecture / Arquitectura
3. Prerequisites / Requisitos Previos
4. Phase 1 — Suricata IDS Setup
5. Phase 2 — ELK Stack (Elasticsearch + Kibana + Filebeat)
6. Phase 3 — Firewall Mock (Flask API)
7. Phase 4 — ngrok Tunnel
8. Phase 5 — n8n SOAR Workflow
9. Phase 6 — Telegram Bot
10. Phase 7 — Suricata Watcher (Python)
11. End-to-End Test / Prueba Completa
12. Troubleshooting / Solución de Problemas
13. Startup Commands / Comandos de Arranque

1. Overview / Descripción del Proyecto

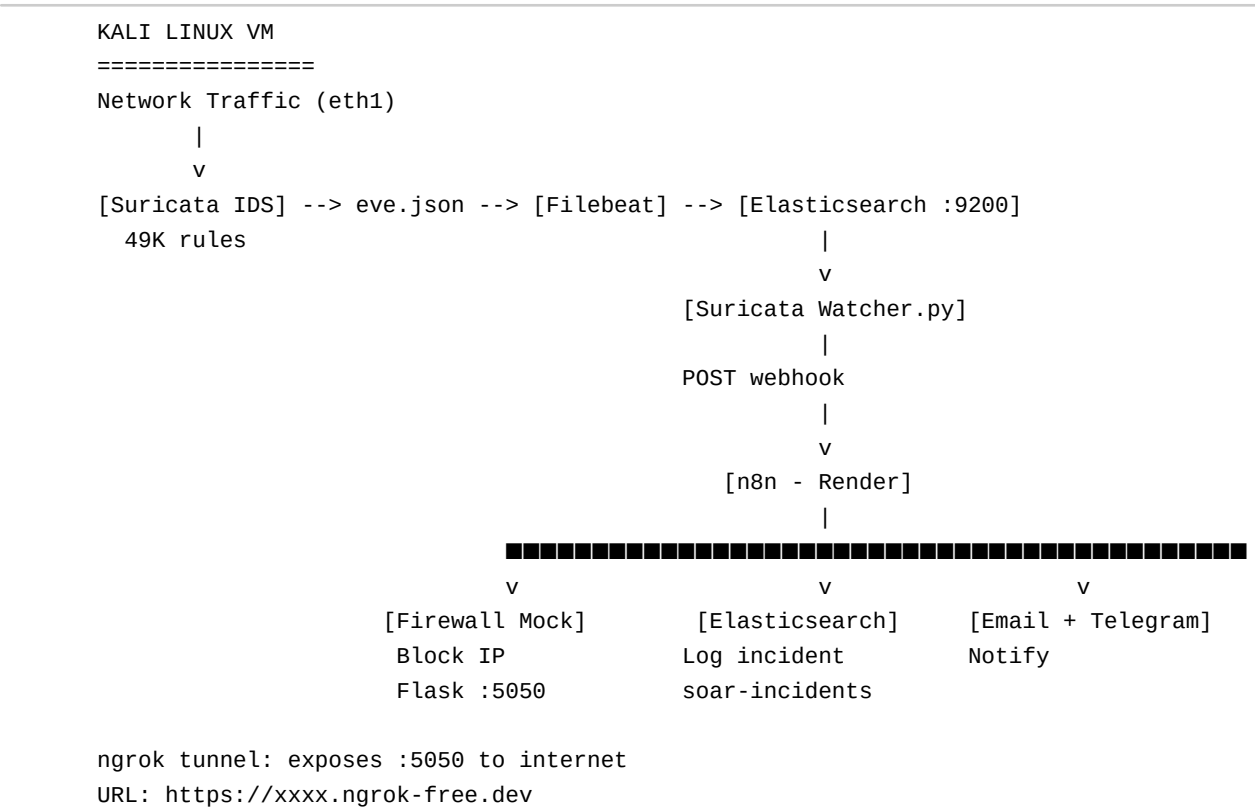
[EN] A fully functional SOC automation lab built on a Kali Linux VM. The system monitors network traffic in real-time using Suricata IDS (49,000+ rules), indexes events into Elasticsearch via Filebeat, and triggers automated incident response workflows through n8n — including IP blocking, incident logging, email alerts, and Telegram notifications.

[ES] Laboratorio SOC funcional desplegado en una VM Kali Linux. El sistema monitoriza el tráfico de red en tiempo real con Suricata IDS (+49,000 reglas), indexa eventos en Elasticsearch mediante Filebeat, y dispara flujos de respuesta automática a incidentes a través de n8n — incluyendo bloqueo de IPs, registro de incidentes, alertas por email y notificaciones por Telegram.

Automated Response Pipeline:

- Suricata detects threat → writes alert to /var/log/suricata/eve.json
- Filebeat indexes alert → pushes to Elasticsearch
- Python Watcher polls Elasticsearch every 60s for new alerts
- n8n webhook receives alert → normalizes fields
- IF high/critical severity: Block IP + Log incident + Email + Telegram
- ELSE: Log to low-severity index

2. Architecture / Arquitectura



Component Table / Tabla de Componentes:

- Suricata IDS** — Network threat detection | *Suricata 8.0.3 + ET Open Rules (49K)*
- Filebeat** — Log shipping to Elasticsearch | *Filebeat 8.19*
- Elasticsearch** — Event storage + search | *Elasticsearch 8.x*
- Kibana** — Visualization dashboard | *Kibana 8.x*
- Suricata Watcher** — Alert polling script | *Python 3*
- n8n** — SOAR orchestration | *n8n (Render cloud)*
- Firewall Mock** — IP blocking simulation | *Flask (Python)*
- ngrok** — Expose local port to internet | *ngrok free tier*
- Mailtrap** — Email alert testing | *SMTP sandbox*
- Telegram Bot** — Instant mobile notifications | *Telegram API*

3. Prerequisites / Requisitos Previos

Hardware / Software:

- VirtualBox with Kali Linux VM (4GB RAM recommended / recomendado)
- 2 network interfaces: NAT (eth0) + Bridged/Host-Only (eth1)
- Internet connection from the VM
- Windows/Mac host for PowerShell testing

Accounts needed / Cuentas necesarias:

- ngrok — <https://ngrok.com> (free tier)
- n8n — <https://n8n.io> (Render free tier)
- Mailtrap — <https://mailtrap.io> (free sandbox)
- Telegram — mobile app + @BotFather to create bot

4. Phase 1 — Suricata IDS Setup

4.1 Install Suricata / Instalar Suricata

```
sudo apt update && sudo apt install suricata suricata-update -y
```

4.2 Update Rules / Actualizar Reglas

Download 49,000+ Emerging Threats rules / Descargar +49,000 reglas Emerging Threats:

```
sudo suricata-update
```

4.3 Configure Network Interface / Configurar Interfaz de Red

Find which interface carries real traffic / Encontrar qué interfaz tiene tráfico real:

```
ip addr show
# Look for your LAN IP, usually eth1 in bridged VMs
# Busca tu IP de red local, normalmente eth1 en VMs bridged
```

4.4 Fix systemd Override / Corregir Override de systemd

[CRITICAL] The default override uses -S analyst.rules which ignores the yaml config. Fix it:

```
sudo nano /etc/systemd/system/suricata.service.d/override.conf
```

Set content to / Establecer contenido a:

```
[Service]
ExecStart=
ExecStart=/usr/bin/suricata -c /etc/suricata/suricata.yaml -i eth1 --pidfile /var/run/suricata/suricata.pid
PIDFile=/var/run/suricata/suricata.pid
Restart=on-failure
```

4.5 Configure suricata.yaml

Ensure rule-files section points to suricata.rules / Asegurar que rule-files apunta a suricata.rules:

```
# Around line 2308 in /etc/suricata/suricata.yaml
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
```

[EN] Indentation must be exactly 0 spaces for default-rule-path and rule-files, 2 spaces for list items.

[ES] La indentación debe ser exactamente 0 espacios para default-rule-path y rule-files, 2 espacios para los elementos de lista.

4.6 Start and Verify / Iniciar y Verificar

```
sudo systemctl daemon-reload
sudo systemctl restart suricata
sleep 15
sudo suricatasc -c "ruleset-stats"
# Expected output / Salida esperada:
```

```
# {"message":[{"id":0,"rules_loaded":49023,...}], "return":"OK"}
```

4.7 Add Custom Rules / Añadir Reglas Personalizadas (Optional)

Create `/var/lib/suricata/rules/analyst.rules` with custom detections:

```
alert icmp any any -> $HOME_NET any (msg:"LOCAL SCAN ICMP Echo Request"; itype:8; icode:0; classtype:icmp-echo; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"LOCAL ALERT SSH Brute Force"; flow:to_server,established; threshold:5,trackers:1; classtype:ssh-brute; sid:1000002; rev:1;)
alert http any any -> $HOME_NET any (msg:"LOCAL ALERT SQL Injection"; http.uri; content:"union select"; classtype:sql-injection; sid:1000003; rev:1;)
```

Then add to `suricata.yaml` rule-files section: - **analyst.rules**

5. Phase 2 — ELK Stack Setup

5.1 Install Elasticsearch

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/debian/apt/pool elasticsearch 7.x" | sudo tee /etc/apt/sources.list.d/elasticsearch.list
sudo apt update && sudo apt install elasticsearch -y
```

Disable security for lab / Deshabilitar seguridad para el lab:

```
sudo nano /etc/elasticsearch/elasticsearch.yml
# Add / Añadir:
xpack.security.enabled: false
xpack.security.http.ssl.enabled: false

sudo systemctl enable --now elasticsearch
# Verify / Verificar:
curl http://localhost:9200
```

5.2 Install Kibana

```
sudo apt install kibana -y
sudo nano /etc/kibana/kibana.yml
# Set / Configurar:
server.host: "0.0.0.0"
elasticsearch.hosts: ["http://localhost:9200"]

sudo systemctl enable --now kibana
# Access at / Acceder en: http://localhost:5601
```

5.3 Install and Configure Filebeat

```
sudo apt install filebeat -y
```

Configure /etc/filebeat/filebeat.yml to read Suricata eve.json:

```
filebeat.inputs:
- type: filestream
  id: suricata-eve
  enabled: true
  paths:
  - /var/log/suricata/eve.json
  parsers:
  - ndjson:
    target: ""
    add_error_key: true

output.elasticsearch:
  hosts: ["http://localhost:9200"]
  index: "filebeat-%[agent.version]}-%[+yyyy.MM.dd}"

sudo systemctl enable --now filebeat
# Verify index exists / Verificar índice:
curl http://localhost:9200/_cat/indices?v | grep filebeat
```


6. Phase 3 — Firewall Mock API (Flask)

Create ~/soar-lab/firewall_api_mock.py:

```
from flask import Flask, request, jsonify
import requests, json
from datetime import datetime

app = Flask(__name__)
blocked_ips = {}
AUTH_TOKEN = "CAMBIAR_POR_TU_TOKEN_FIREWALL"

@app.route('/block-ip', methods=['POST'])
def block_ip():
    token = request.headers.get('X-Auth-Token')
    if token != AUTH_TOKEN:
        return jsonify({"error": "Unauthorized"}), 401
    data = request.json
    ip = data.get('ip')
    reason = data.get('reason', 'No reason provided')
    blocked_ips[ip] = {"reason": reason, "timestamp": str(datetime.now())}
    print(f"[FIREWALL] Blocked IP: {ip} | Reason: {reason}")
    return jsonify({"status": "success", "ip": ip, "action": "blocked"})

@app.route('/blocked', methods=['GET'])
def list_blocked():
    return jsonify({"blocked_ips": blocked_ips, "count": len(blocked_ips)})

# Proxy to Elasticsearch (avoids needing second ngrok tunnel)
@app.route('/elastic/<path:path>', methods=['GET', 'POST', 'PUT'])
def elastic_proxy(path):
    url = f"http://localhost:9200/{path}"
    resp = requests.request(method=request.method, url=url,
        headers={"Content-Type": "application/json"},
        data=request.get_data())
    return resp.content, resp.status_code

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5050, debug=True)

pip3 install flask requests
python3 ~/soar-lab/firewall_api_mock.py
```

Note / Nota: Keep this running in a dedicated terminal. Use a second terminal for other commands.

7. Phase 4 — ngrok Tunnel

7.1 Install ngrok

```
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc &&
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list &&
sudo apt update && sudo apt install ngrok -y
```

7.2 Authenticate

```
ngrok config add-authtoken YOUR_TOKEN_FROM_NGROK_DASHBOARD
```

7.3 Start tunnel

```
ngrok http 5050
# Note the URL: https://xxxx.ngrok-free.dev
# This URL must be updated in n8n nodes when it changes
```

Important / Importante: Free tier ngrok URL changes every time you restart the agent. Update all n8n HTTP nodes with the new URL when this happens.

Tip: Always add header **ngrok-skip-browser-warning: 1** to all HTTP requests going through ngrok.

8. Phase 5 — n8n SOAR Workflow

8.1 Create n8n account on Render

- Go to <https://render.com> and create free account
- Deploy n8n as a Web Service using the official n8n Docker image
- Note your n8n URL: <https://your-app.onrender.com>
- Log into n8n and create new workflow named 'SOAR Suricata'

8.2 Workflow Nodes — Build in this order:

Node 1 — Webhook (Trigger)

Method: POST
Path: suricata-alert
Response: Last Node
Response Body: Alerta recibida correctamente

Node 2 — Normalizar Alerta (Code (JavaScript))

```
const body = $input.first().json.body || $input.first().json;  
return [{  
  json: {  
    rule_name: body.rule_name || body['rule.name'] || 'Unknown',  
    severity: body.severity || 'medium',  
    source_ip: body.source_ip || body.src_ip || '0.0.0.0',  
    destination_ip: body.destination_ip || body.dest_ip || '0.0.0.0',  
    timestamp: body.timestamp || new Date().toISOString(),  
    event_module: body.event_module || 'suricata',  
    event_action: body.event_action || 'alert'  
  }  
}];
```

Node 3 — If (Alta Severidad) (IF)

Condition 1: `{{ $json.severity }}` equals high
Condition 2: `{{ $json.severity }}` equals critical
Combine: OR | Convert Types: ON

Node 4 — Bloquear IP (Firewall) (HTTP Request [TRUE branch])

Method: POST
URL: <https://YOUR-NGROK-URL/block-ip>
Header: X-Auth-Token: CAMBIAR_POR_TU_TOKEN_FIREWALL
Header: ngrok-skip-browser-warning: 1
Body (JSON): `{"ip": "{{ $('Normalizar Alerta').item.json.source_ip }}", "reason": "{{ $('Normalizar Alerta').item.json.severity }}"}`

Node 5 — Registrar Incidente (Alta) (HTTP Request [TRUE branch])

Method: POST
URL: https://YOUR-NGROK-URL/elastic/soar-incidents/_doc
Header: ngrok-skip-browser-warning: 1
Body: all fields from Normalizar Alerta node

Node 6 — Notificar Email (Send Email [TRUE branch])

SMTP Credentials: Mailtrap (sandbox.smtp.mailtrap.io:2525)
From/To: your@email.com
Subject: [SOAR ALERT] {{ \$('Normalizar Alerta').item.json.rule_name }}
Body: HTML with all alert fields

Node 7 — Notificar Telegram (Telegram [TRUE branch])

Credential: your bot token
Chat ID: your chat ID
Text: Alert message with emoji and fields

Node 8 — Registrar Evento (Baja) (HTTP Request [FALSE branch])

Method: POST
URL: https://YOUR-NGROK-URL/elastic/soar-low-severity/_doc
Body: all fields from Normalizar Alerta node

8.3 Activate Workflow

Click the toggle in top-right to set workflow as Active / Hacer clic en el toggle arriba a la derecha para activar el workflow.

9. Phase 6 — Telegram Bot

9.1 Create Bot

- Open Telegram, search @BotFather
- Send /newbot
- Choose name: SOAR Alert Bot
- Choose username (must end in 'bot'): soar_alert_XXXX_bot
- Save the token provided by BotFather

9.2 Get Chat ID

```
# 1. Find your bot in Telegram and send /start
# 2. Open in browser:
https://api.telegram.org/botYOUR_TOKEN/getUpdates
# 3. Find: "chat":{"id":XXXXXXXX}
# That number is your Chat ID
```

9.3 Test Bot

```
curl "https://api.telegram.org/botYOUR_TOKEN/sendMessage?chat_id=YOUR_CHAT_ID&text=Test+SOAR+Alert"
```

9.4 Telegram Node Message Template

```
■ Alerta SOAR
Regla: {{ $('Normalizar Alerta').item.json.rule_name }}
Severidad: {{ $('Normalizar Alerta').item.json.severity }}
IP Origen: {{ $('Normalizar Alerta').item.json.source_ip }}
IP Destino: {{ $('Normalizar Alerta').item.json.destination_ip }}
Timestamp: {{ $('Normalizar Alerta').item.json.timestamp }}
```

10. Phase 7 — Suricata Watcher (Python)

Create ~/soar-lab/suricata_watcher.py:

```
import requests
import json
import time
from datetime import datetime, timedelta, timezone

ELASTICSEARCH_URL = "http://localhost:9200"
INDEX = ".ds-filebeat-*"
N8N_WEBHOOK = "https://YOUR-N8N-URL/webhook/suricata-alert"
POLL_INTERVAL = 60 # seconds

def search_alerts():
    now = datetime.now(timezone.utc)
    since = (now - timedelta(minutes=2)).strftime('%Y-%m-%dT%H:%M:%SZ')

    query = {
        "query": {
            "bool": {
                "must": [
                    {"match": {"event_type": "alert"}},
                    {"range": {"@timestamp": {"gte": since}}}
                ]
            }
        },
        "sort": [{"@timestamp": "desc"}],
        "size": 10
    }

    response = requests.post(
        f"{ELASTICSEARCH_URL}/{INDEX}/_search",
        headers={"Content-Type": "application/json"},
        data=json.dumps(query)
    )
    return response.json().get("hits", {}).get("hits", [])

def send_to_n8n(alert):
    source = alert.get("_source", {})
    alert_data = source.get("alert", {})

    payload = {
        "rule_name": alert_data.get("signature", "Unknown"),
        "severity": ["critical", "high", "medium", "low"][min(alert_data.get("severity", 3)-1, 3)],
        "source_ip": source.get("src_ip", "0.0.0.0"),
        "destination_ip": source.get("dest_ip", "0.0.0.0"),
        "event_module": "suricata",
        "event_action": "alert",
        "timestamp": source.get("timestamp", datetime.now().isoformat())
    }

    response = requests.post(N8N_WEBHOOK,
        headers={"Content-Type": "application/json"},
```

```

        data=json.dumps(payload))
    return response.status_code

if __name__ == "__main__":
    print("[Watcher] Starting Suricata Alert Watcher...")
    while True:
        alerts = search_alerts()
        if alerts:
            for alert in alerts:
                status = send_to_n8n(alert)
                sig = alert["_source"].get("alert", {}).get("signature", "Unknown")
                print(f"[Alerta enviada -> n8n ({status})] {sig}")
            else:
                print(f"[{datetime.now().strftime('%H:%M:%S')}] Sin alertas nuevas")
        time.sleep(POLL_INTERVAL)

python3 ~/soar-lab/suricata_watcher.py

```

11. End-to-End Test / Prueba Completa

11.1 Manual Test via PowerShell / Prueba Manual

```
Invoke-RestMethod -Uri "https://YOUR-N8N-URL/webhook/suricata-alert" `
  -Method POST -ContentType "application/json" -Body '{
  "rule_name": "ET SCAN Nmap",
  "severity": "high",
  "source_ip": "192.168.1.100",
  "destination_ip": "10.0.0.1",
  "event_module": "suricata",
  "event_action": "alert",
  "timestamp": "2026-03-12T12:00:00Z"
}'
```

11.2 Real Traffic Test / Prueba con Tráfico Real

```
# Terminal 1 - Monitor alerts
sudo tail -f /var/log/suricata/eve.json | grep 'event_type.*alert'

# Terminal 2 - Generate malicious traffic
curl http://testmyids.org/uid/index.html
# or / o:
sudo nmap -sS 192.168.1.134
```

11.3 Verify Results / Verificar Resultados

```
# Check blocked IPs
curl http://localhost:5050/blocked | python3 -m json.tool

# Check Elasticsearch incidents
curl 'http://localhost:9200/soar-incidents/_search?pretty' | grep source_ip

# Check Suricata rules loaded
sudo suricatasc -c "ruleset-stats"
# Expected: rules_loaded: 49023
```

11.4 Expected Results / Resultados Esperados

- n8n webhook responds 'Alerta recibida correctamente'
- IP appears in /blocked endpoint
- Incident logged in soar-incidents Elasticsearch index
- Email received in Mailtrap inbox
- Telegram message received on mobile
- n8n Executions shows green workflow run

12. Troubleshooting / Solución de Problemas

Problem: Suricata loads only 11 rules

Check systemd override: `sudo systemctl cat suricata | grep ExecStart`
Remove `-S analyst.rules` flag from `override.conf`
Verify `suricata.yaml` `rule-files` section points to `suricata.rules`

Problem: Suricata fails to start after yaml edit

Run: `sudo suricata -T -c /etc/suricata/suricata.yaml 2>&1 | grep -i error`
Check indentation in `rule-files` section (must be 0 spaces for keys, 2 for list items)
Restore backup: `sudo cp /etc/suricata/suricata.yaml.dpkg-dist /etc/suricata/suricata.yaml`

Problem: ngrok URL changes / n8n can't reach firewall

Update ALL HTTP Request nodes in n8n with new ngrok URL
Always add header: `ngrok-skip-browser-warning: 1`
Consider upgrading to ngrok paid plan for static URL

Problem: Port 5050 already in use

`sudo fuser -k 5050/tcp`

Problem: Firewall mock stops working

Check if process is still running: `ps aux | grep firewall`
Restart: `python3 ~/soar-lab/firewall_api_mock.py`

Problem: Watcher shows 'Sin alertas nuevas' always

Verify `event_type` field: `curl localhost:9200/_search -d '{"query":{"match":{"event_type":"alert"}}}'`
Check timestamp timezone mismatch
Generate fresh traffic: `curl http://testmynids.org/uid/index.html`

Problem: n8n Render sleeps (cold start)

First request may take 30-60 seconds to wake up
Send a test request and wait before checking

Problem: Elasticsearch not accessible

`sudo systemctl status elasticsearch`
`curl http://localhost:9200`
Check `xpack.security.enabled: false` in `elasticsearch.yml`

13. Startup Commands / Comandos de Arranque

Run in this exact order / Ejecutar en este orden exacto:

Step 1 — Verify core services are running:

```
sudo systemctl status elasticsearch kibana filebeat suricata
# All should show: active (running)
```

Step 2 — Start Firewall Mock (dedicated terminal):

```
cd ~/soar-lab && python3 firewall_api_mock.py
```

Step 3 — Start ngrok tunnel (new terminal):

```
ngrok http 5050
# Copy the https://xxxx.ngrok-free.dev URL
```

Step 4 — Update *n8n* with new *ngrok* URL (if changed):

Go to n8n → edit each HTTP Request node → update URL → save → publish

Step 5 — Start Suricata Watcher (new terminal):

```
python3 ~/soar-lab/suricata_watcher.py
```

Quick Status Check / Verificación Rápida de Estado

```
# Services
sudo systemctl is-active elasticsearch kibana filebeat suricata
```

```
# Rules loaded
sudo suricatasc -c "ruleset-stats"
```

```
# Recent alerts in Elasticsearch
curl -s 'localhost:9200/.ds-filebeat-*/_count' -H 'Content-Type: application/json' -d '{"query":{"
```

```
# Blocked IPs
curl http://localhost:5050/blocked
```

■ Lab Complete / Lab Completado

Full pipeline operational / Pipeline completo operativo:

- Suricata IDS — 49,023 rules active
- Filebeat → Elasticsearch — events indexed
- Kibana — visualization ready
- Firewall Mock — IP blocking operational
- ngrok — tunnel active
- n8n — SOAR workflow published
- Email (Mailtrap) — notifications working
- Telegram Bot — instant mobile alerts
- Suricata Watcher — automated pipeline active
- End-to-end response time: < 60 seconds

Marcos — @Markussella | github.com/Markussella/blue-team-lab

Built as a hands-on cybersecurity portfolio project — Blue Team / SOC Analyst